

Trabajo final - Telemetría hidrometeorológica en streaming

Documento técnico: Kafka, Apache Beam sobre Flink y Marimo

abarchello · Streaming de datos y sus aplicaciones · MIAAD FP-UNA

Septiembre 2026

1. Problema y usuarios

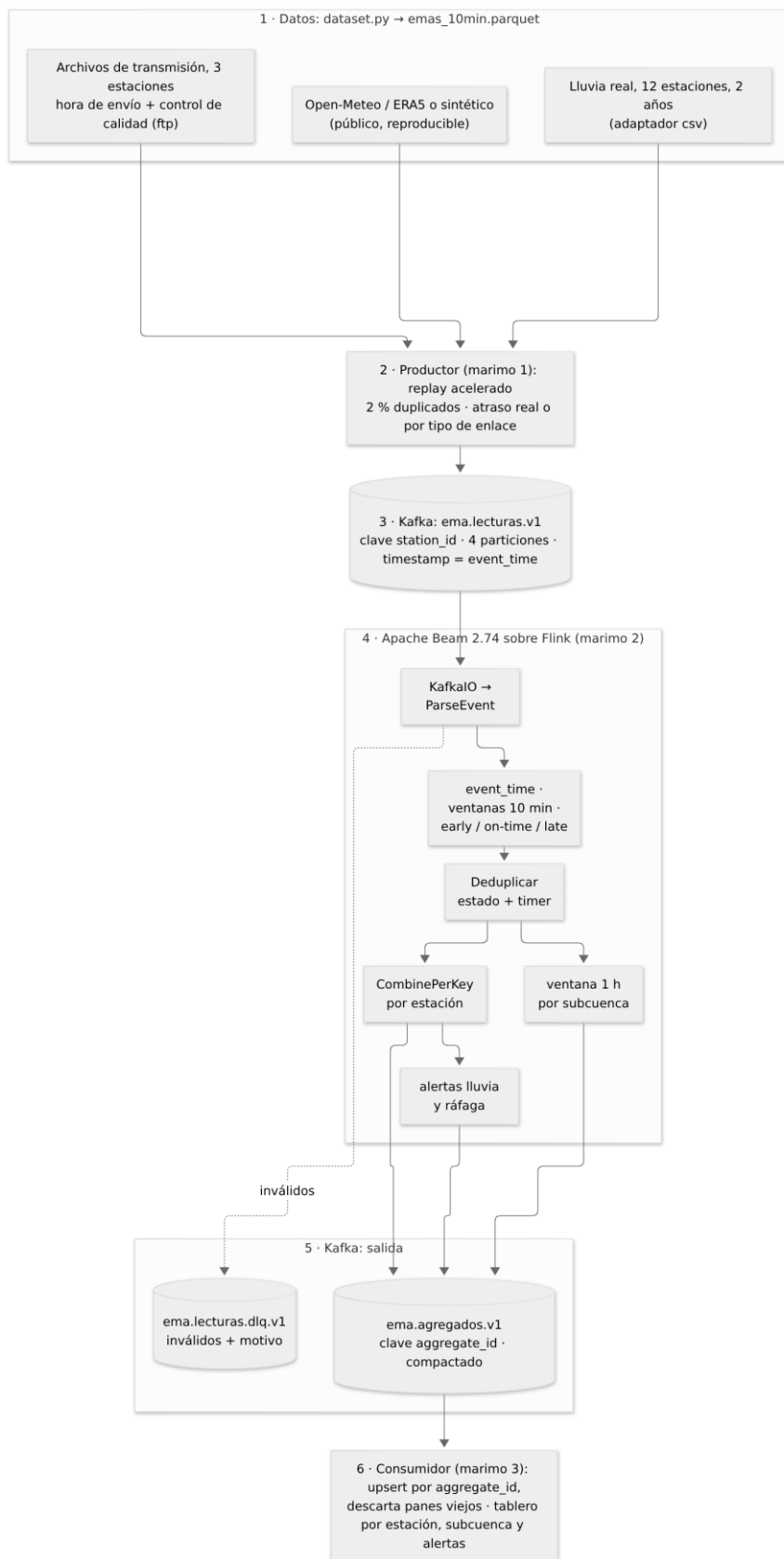
Las estaciones meteorológicas automáticas de Alto Paraná y Canindeyú miden cada 10 minutos lluvia, temperatura, humedad, presión, viento y radiación. Durante un evento de lluvia, quien vigila la cuenca necesita saber cuánto llovió en los últimos 10 minutos en cada estación y en la última hora en cada subcuenca, cuántas estaciones están reportando, y recibir una alerta cuando se pasa un umbral (10 mm en 10 minutos, o ráfagas de 20 m/s).

Los datos no llegan limpios: los enlaces GPRS juntan lecturas y las mandan en ráfaga, los satelitales se atrasan, los dataloggers reintentan y duplican, y cada tanto aparece un valor imposible. Un proceso por lotes nocturno resuelve todo eso, pero llega tarde para decidir. El pipeline da una estimación en segundos, un resultado firme al cerrar cada ventana y una corrección por cada dato tardío, sin contar nada dos veces.

Métricas que habilita: lluvia por estación cada 10 minutos, lluvia areal por subcuenca cada hora (promedio de las estaciones, con cuántas entraron), alertas de lluvia intensa y de ráfaga, y la cantidad de lecturas inválidas.

2. Arquitectura

- **Datos** (`dataset.py`): cinco fuentes con el mismo esquema. Las públicas (Open-Meteo/ERA5 y un generador sintético) hacen el proyecto reproducible; las reales (sección 3) no se publican.
- **Productor** (`producer.py`, notebook 1): reproduce un período histórico como si llegara en vivo, acelerado 60 veces, con un 2 % de duplicados y el atraso de cada lectura (real si se conoce, si no según el tipo de enlace).
- **Kafka 4.1.1** (KRaft): tópico de entrada, tópico de salida compactado y DLQ.
- **Pipeline** (`pipeline.py`, `transforms.py`, notebook 2): Apache Beam 2.74 con `ReadFromKafka` (KafkaIO), sobre Flink 1.19 con el PortableRunner, en streaming. Valida, deduplica, agrega por ventana y detecta alertas.
- **Consumidor** (`consumer.py`, notebook 3): dashboard que materializa la salida con upsert por clave.
- **Operación**: Docker Compose levanta todo; `make smoke` prueba el recorrido Kafka → Beam/Flink → Kafka.



Arquitectura del pipeline, de la fuente a la salida

3. Datos reales y replay

Fuente: Itaipú Binacional, División de Embalse. Los datos no están en el repositorio y las estaciones llevan códigos y nombres de ejemplo (ITA01 a ITA12), los mismos de las Tareas 1 y 2.

- **Lluvia de 12 estaciones en dos años** (julio 2024 a junio 2026): 792 653 lecturas. Siete estaciones con más del 93 % de cobertura y cinco con huecos reales, que el pipeline respeta (no emite ventanas sin lecturas).
- **Archivos de transmisión de 3 estaciones** (julio a septiembre 2026): 26 640 lecturas con todas las variables. Una estación manda un archivo por transmisión con la hora de envío, así que para ella conozco el **atraso real** de cada lectura, y el replay lo reproduce tal cual. Se copian a una base local (DuckDB) y se adaptan con control de calidad por variable: los códigos de estado y centinelas quedan vacíos, y un valor fuera de rango físico (por ejemplo, una presión descalibrada) deja vacía sólo esa variable con la marca en `flag_qc`, sin perder la lluvia de la lectura.

4. Contrato, tópicos y particiones

El evento `ema.lectura` (JSON, `schema_version = 1`) trae `event_id`, `event_type`, `event_time` (UTC), `station_id`, metadatos de la estación, las 8 variables (sólo la lluvia obligatoria), `source` y `flag_qc`. El `event_id` es `ema-<estación>-<event_time>`: un reenvío repite el id y se puede deduplicar sin consultar al productor. Un cambio de esquema se publica en un tópico `.v2` y convive con el `.v1` mientras migran los consumidores.

Tópico	Clave	Particiones	Configuración
<code>ema.lecturas.v1</code>	<code>station_id</code>	4	<code>timestamp de Kafka = event_time</code>
<code>ema.agregados.v1</code>	<code>aggregate_id</code>	4	<code>cleanup.policy=compact</code>
<code>ema.lecturas.dlq.v1</code>	<code>invalid</code>	4	<code>{error, payload}</code> : motivo y mensaje original

La clave de entrada es la estación: el orden que importa es el de las lecturas de una misma estación, y ése se conserva dentro de su partición. Con 12 estaciones tocan 2 o 3 por partición; todas emiten con la misma cadencia, así que no hay claves calientes. Con 4 particiones el paralelismo puede subir de 2 a 4 sin reparticionar.

La salida trae `aggregate_id = metric_type|window_start|dimension_id`, `metric_type` (`estacion_10min`, `subcuenca_1h`, `alerta`), la ventana, el pane (`pane_index`, `pane_timing`, `is_first`, `is_last`) y las métricas.

5. Tiempo de evento, duplicados y semántica

Decisión	Valor	Por qué
Timestamp	<code>event_time</code> (política <code>create_time</code> de KafkaIO)	La lluvia pertenece al intervalo en que cayó; un replay da lo mismo.
Ventanas	fijas de 10 min (estación) y 1 h (subcuenca)	Frecuencia de las estaciones y escala de la lluvia areal.
Trigger	early cada 10 s, on-time, late por cada dato	Estimación, resultado firme y corrección.
Acumulación	<code>ACCUMULATING</code>	Cada pane trae el total: permite upsert.
Lateness	20 min	Ver calibración abajo.
Deduplicación	<code>SetState</code> por estación y ventana, timer en fin + lateness	Horizonte = ventana + 20 min; el estado no crece.

Calibración de la lateness con llegadas reales. En la estación con hora de envío, el atraso tiene mediana de 0,9 minutos y p95 de 2,8, pero el p99 es de 294,6 minutos y el máximo de 874: son cortes de enlace, tras los cuales la estación manda lo acumulado. Con 20 minutos quedan afuera 172 lecturas (1,95 %); con 300 minutos, 86 (0,97 %). Subir la lateness al p99 multiplica por diez el tiempo que cada ventana mantiene estado para recuperar un punto porcentual, así que mantengo 20 minutos y dejo lo posterior a un corte para un reproceso aparte.

Semántica por tramo. No afirmo exactly-once de punta a punta:

- Productor → Kafka: idempotente (`enable.idempotence=true, acks=all`).
- Kafka → Beam: KafkaIO confirma offsets con `enable.auto.commit`, aparte de los checkpoints de Flink (cada 30 s): si el job se reinicia, algunas lecturas se reprocesan. Es **at-least-once**.
- Beam → `ema.agregados.v1`: at-least-once, con productor idempotente.
- Vista final: **idempotente**. El consumidor hace upsert por `aggregate_id`, descarta panes con `pane_index` menor al que tiene, y el tópico es compactado: reprocesar o republicar no cambia el resultado visible.

En producción confirmaría los offsets al terminar cada checkpoint, para que estado y offsets se restauren juntos.

6. Pruebas y evidencia

- **22 pruebas** (`make test`): contrato (ids deterministas, rechazo de inválidos), `CombineFn`, pipeline en batch (deduplicación, agregados, alertas, DLQ), adaptadores de datos con su control de calidad, productor (atraso real y replay por lotes) y consumidor (upsert, panes viejos y pane vacío de cierre). Cuatro usan `TestStream` y avanzan el watermark a mano: pane LATE acumulado después del `ON_TIME`, dato más tardío que la lateness descartado sin tocar el total, duplicado ignorado y `pane_timing` publicado con su nombre.
- **Smoke test** (`make smoke`): Kafka → Beam sobre Flink → Kafka con Docker, código de salida 0 (`docs/evidencia_smoke.txt`).
- **Referencias con datos reales** (`scripts/run_local.py`, la misma lógica en batch):

Dataset	Lecturas	Duplicados	Agregados de estación	De subcuenca	Alertas
Lluvia (200 000 primeras)	200 000	3 960	200 000	24 314	27
Transmisión (3 estaciones)	26 640	507	26 640	4 443	5

En ambos casos hay un agregado de estación por lectura: los duplicados no cambian ningún total, y las lecturas atrasadas cambian el orden de llegada pero no la ventana. Un cálculo aparte en pandas, sin Beam, da la misma lluvia por estación, los mismos pares de subcuenca y hora y las mismas alertas. Las 2 entradas a la DLQ son los inválidos que agrega el script.

- **En streaming sobre Flink**, un día de lluvia con 12 estaciones (1 584 lecturas), en dos modos. Replay acelerado a 600×: 1 618 eventos con 34 duplicados y 1 122 atrasados, panes EARLY cada 10 s con una sola lectura por ventana, alertas, agregados por subcuenca y un evento roto en la DLQ con su motivo. Replay por lotes con las fechas originales: las 1 584 ventanas cierran `ON_TIME` en menos de 10 s, con 144 agregados por subcuenca y 7 alertas, todos firmes. Hacen falta los dos porque la política `CreateTime` de KafkaIO fija el watermark en el reloj cuando los eventos están en el futuro, que es lo que produce un replay desplazado a “ahora” y acelerado (lo verifiqué en el código de la jar con `javap`; el laboratorio de la clase 7 tiene el mismo comportamiento). LATE y descarte quedan probados con `TestStream`.

7. Límites, supuestos y mejoras

- Lo que llega después de fin de ventana más 20 minutos se pierde y sólo queda el contador del runner. Mejora: tópico de auditoría para los `too_late` (propuesto en la Tarea 2) y reproceso por lotes de esas ventanas.
- La entrada es at-least-once; la corrección visible depende del upsert idempotente.
- Con `CreateTime` de KafkaIO un replay acelerado no cierra ventanas en vivo: la demo muestra el cierre con el replay por lotes y las estimaciones con el acelerado. Panes LATE en Flink pedirían un replay a velocidad real (más de media hora) o una política de watermark propia, que el SDK de Python no permite definir para KafkaIO.

- Al expirar una ventana, Flink emite un pane de cierre vacío: el pipeline lo descarta y el consumidor lo ignora (hallazgo del ensayo).
- El mapeo de códigos de sensor a variables lo deduje de las unidades de los encabezados de los archivos diarios; no es una tabla oficial. Sólo una estación informa la hora de envío.
- La referencia en DirectRunner no procesa los dos años enteros (no entran en memoria); por eso usa las primeras 200 000 lecturas.
- El dashboard vive en memoria y relee el tópico compactado en cada sesión; las alertas comparten tópico con los agregados.
- Pendiente: detección de anomalías en streaming sobre `ema.agregados.v1` (clase 8), por ejemplo una estación cuya lluvia se aleja de su subcuenca.

8. Integrantes y uso de IA

Trabajo individual (`abarchello`): caso de uso, contrato, replay, pipeline, pruebas, infraestructura, dashboard y documentación. Usé herramientas de IA generativa como apoyo para redactar y revisar textos, escribir y revisar código y pruebas, y contrastar decisiones con lo visto en clase. Las decisiones, la ejecución y la verificación son mías, y cada cifra sale de correr el código del repositorio.